COM2016B – Programming with Python
University of Exeter
6th April 2026
Module Leader: Milto Miltiadou

Implement a command line version of the board game "Cluedo"

Joshua Bowley | jb1753@exeter.ac.uk

I acknowledge the following uses of GenAI tools in this assessment:

- ✓ I have used GenAI tools to:
  - ✓ develop ideas.
  - ☐ assist with research or gathering information.
  - ✓ help me understand key theories and concepts.
  - ☐ identify trends and themes as part of my data analysis.
  - ✓ suggest a plan or structure for my assessment.
  - ✓ give me feedback on a draft.
  - ✓ generate images, figures or diagrams.
  - ✓ proofread and correct grammar or spelling errors.
  - ☐ generate citations or references.
  - ✓ Other: Generate code from pseudocode and design notes, generate pytest unit tests, generate and review docstrings to PEP 257 conventions.
- ☐ I have not used any GenAI tools in preparing this assessment.

I declare that I have referenced all use of GenAI outputs within my assessment in line with the University referencing guidelines. I certify that all material in this dissertation which is not my own has been identified.

Links

GitHub Repository: `https://github.com/joshbowley/com2026_cluedo`

Video Demonstration: COM2016_B_Project_Video_Josh_Bowley.mp4

Game Modifications

No game modifications applied.

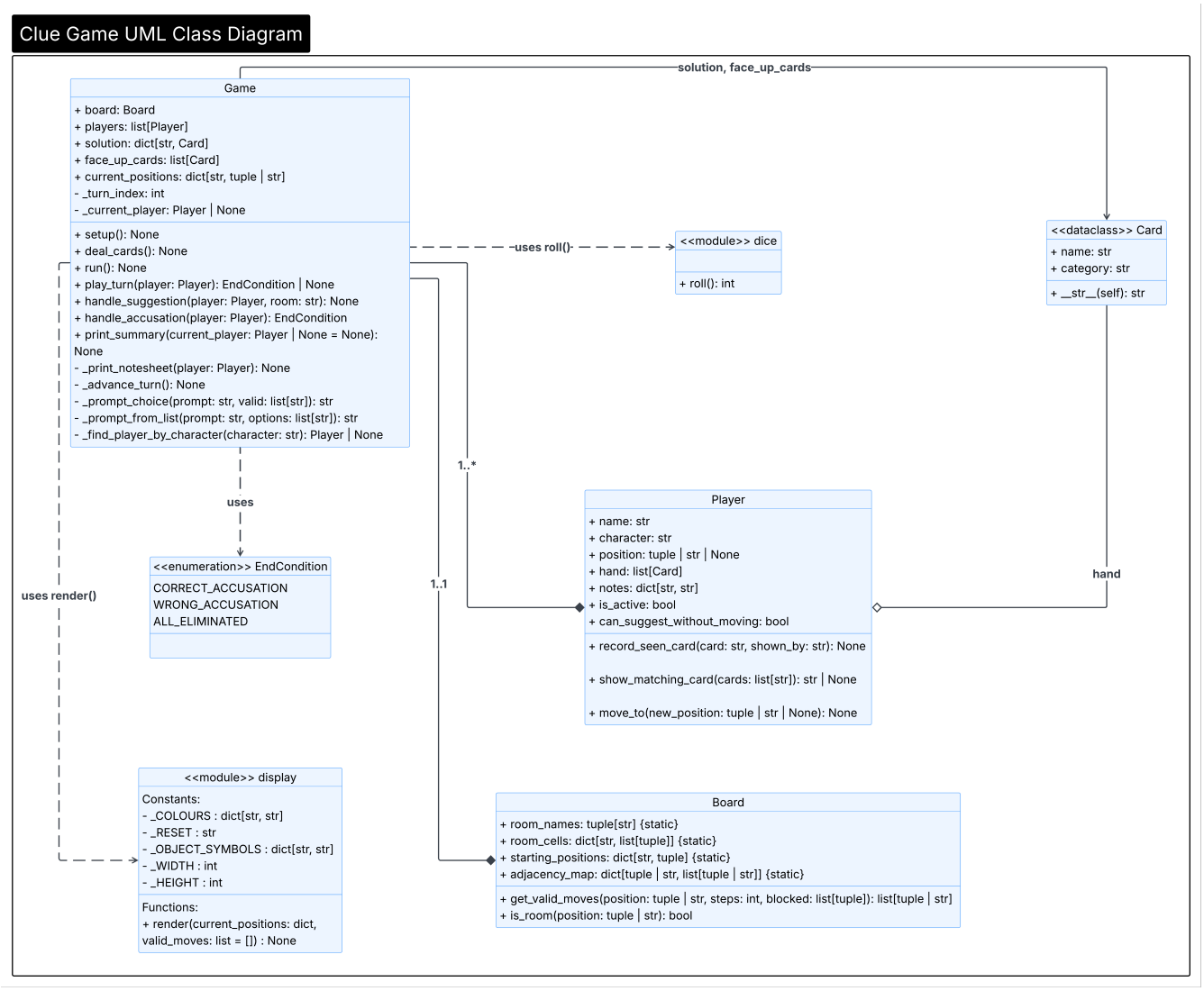


Figure 1: Cluedo Game UML Class Diagram

Biggest Challenges

Challenge 1. Representing the board as a data structure:

The board contains two fundamentally different position types — corridor coordinate tuples and named rooms — which needed to coexist in a single movement system. I used a mixed-key adjacency map dictionary where corridor positions are `(row, col)` tuples and rooms are strings, allowing player position to be typed as `tuple | str` and movement to work uniformly across both.

Challenge 2. Implementing valid move generation:

Computing all reachable positions within exactly N steps required a graph traversal algorithm that respected both step limits and the rule that rooms are terminal destinations. I implemented Breadth-First Search (BFS) using Python's `collections.deque`, exploring positions layer by layer, while rooms are treated as terminal: BFS enters them but doesn't expand further.

Challenge 3. Building an accurate adjacency map:

Encoding the full 25-by-24 corridor layout of the 1990 Waddington's board as a dictionary by hand was error-prone, and several non-reciprocal connections were introduced. I wrote a `check_reciprocity()` function that verified every A to B connection had a corresponding B to A, and used it iteratively to identify and fix all errors.

Challenge 4. Rendering player positions correctly when players are in rooms:

The `render()` function crashed with a `TypeError` when a player's position was a room string, because the grid indexing assumed all positions were integer tuples. I added an `isinstance(position, tuple)` guard so only corridor-based players are drawn as tokens, since players in rooms are already represented by the room's coloured cells.

Challenge 5. Making the CLUE sheet accessible at every input point:

The `p` key initially only triggered the game summary at the move selection prompt, not during suggestions or accusations. I added a `_current_player` attribute set at the start of each turn in `play_turn()`, and updated both `_prompt_choice()` and `_prompt_from_list()` to call `print_summary(self._current_player)`, so the CLUE sheet is accessible at every input prompt without duplicating code.

OOP Design Choices

Choice 1. `@dataclass` for `Card`:

`Card` is a small value object (name and category) with no internal state that changes over time. Python's `@dataclass` decorator automatically generates `__init__`, `__repr__`, and `__eq__`, meaning I didn't have to write them by hand and adding a new field later is a one line change.

Choice 2. `Enum` for `EndCondition`:

Game endings are a fixed, named set of mutually exclusive states: `CORRECT_ACCUSATION`, `WRONG_ACCUSATION`, and `ALL_ELIMINATED`. Using an `Enum` makes `handle_accusation()` return a typed value rather than a string or integer. Without this, I'd have been comparing strings like `"wrong_accusation"` throughout `run()`, which is fragile.

Choice 3. Class-level constants on `Board`:

The adjacency map, room cells, room names and starting positions are defined as module-level constants and assigned as class attributes on `Board`, so they are shared across all instances and don't change at runtime. This makes my intent clear: these are constants, not a state that belongs to any particular game instance. It also keeps `__init__` empty, showing that `Board` has no per-instance state to set up.

Choice 4. Private methods with leading underscores in `Game`:

Methods such as `_advance_turn()`, `_prompt_choice()`, `_prompt_from_list()`, and `_print_notesheet()` are internal helpers not intended to be called from outside the class. Prefixing them with a single underscore follows PEP 8 convention and makes it obvious which methods are called from outside the class and which aren't.

Choice 5. Separation of rendering from game logic:

The `render()` function lives in its own `display.py` module rather than inside `Game` or `Board`, meaning the board can be rendered from any point in the game by passing only `current_positions` and optionally `valid_moves`, without `display.py` needing to know anything about game rules. It also means the display logic can be modified without touching the game logic at all (for example, adding colour or changing symbols).